

目 录

1. 语义分析测试概览	3
2. C++ 单元测试	3
2.1. 成功测试 (8 项)	3
2.2. 错误测试 (11 项)	4
3. 集成测试 (合法用例)	5
3.1. 多类型变量声明与赋值 (sv01)	5
3.2. 过程调用与 var 参数 (sv07)	6
3.3. 阶乘递归函数 (sv17)	6
3.4. 整数与实数混合运算 (sv19)	7
3.5. 非零起始数组 (sv21)	7
3.6. while 循环内 break (sv11)	8
4. 集成测试 (非法用例)	8
4.1. 未声明变量 (si01)	8
4.2. 变量重复声明 (si02)	9
4.3. 赋值类型不匹配 (si06)	9
4.4. 循环外 break (si11)	9
4.5. 数组下标越界 (si12)	10
4.6. 参数数量不匹配 (si15)	10
4.7. var 参数传字面量 (si17)	10
4.8. div 运算用于实数 (si18)	11
4.9. 过程调用用作表达式值 (si24)	11
4.10. read 读取常量 (si25)	11
4.11. 实数缩窄赋值给整数 (si27)	12
5. 注解后 AST 示例	12
5.1. 基本声明与赋值 (sv01)	12
5.2. 过程调用与 var 参数 (sv07)	13
5.3. 数组访问与边界注解 (sv10)	15
5.4. 递归函数: if-else 与返回值赋值 (sv17)	15
5.5. 错误用例: 类型不匹配时的注解 (si06)	17
6. 自动测试结果	18

7. 测试结果汇总	19
-----------------	----

语义分析单元测试报告

1. 语义分析测试概览

语义分析测试分为两个层次：

1. **C++ 单元测试：**绕过词法/语法分析器，使用 `ASTBuilder` 直接构建 AST，对 `SemanticAnnotator` 进行白盒测试。通过 `test/test_semantic_unit.sh` 编译并运行独立的 C++ 测试程序。共 19 项测试。
2. **集成测试：**使用 `--semantic` 模式运行完整编译流水线（词法 → 语法 → 语义），通过 `.pas` 文件进行端到端语义验证。共 55 个用例（26 合法 + 29 非法）。

2. C++ 单元测试

C++ 单元测试通过 `test/test_semantic_unit.sh` 编译运行 `test/semantic_unit/semantic_unit.cpp`，直接测试符号表和语义分析器的核心逻辑。共 19 项测试，其中 8 项为成功测试（预期无错误），11 项为错误测试（预期特定错误消息）。

2.1 成功测试（8 项）

testValidDeclarationAndAssignment：声明一个 `integer` 类型的变量 `x`，然后赋值为 `1+2`（两个字面量相加）。预期语义分析无错误通过，验证最基本声明和赋值流程。

testIntegerToRealAssignmentAllowed：声明一个 `real` 类型的变量 `r`，然后将整数字面量 `1` 赋值给它。预期语义分析允许整数到实数的隐式拓宽，不产生错误。

testProcedureCallValid：声明过程 `p`（含 `var` 引用参数 `x: integer` 和值参数 `y: real`），调用时传入兼容类型的实参。预期语义分析通过，验证参数类型匹配检查。

testFunctionResultAssignmentValidInsideFunction：声明返回 `integer` 的函数 `f`，函数体内执行 `f := 1`。预期语义分析认可函数内部对函数名的赋值作为合法的返回值设置。

testReadIntoFunctionResultValidInsideFunction: 声明返回 integer 的函数 `getint`，函数体内执行 `read(getint)`，将输入直接读入函数返回值。预期语义分析认可以此模式作为合法的 `read` 左值。

testArrayIndexInBounds: 声明范围 1..3 的数组，使用下标 2 访问。预期语义分析无错误通过，验证数组下标在声明范围内合法。

testBuiltinReadWritePreregistered: 调用 `read(x)` 和 `write(x)` 而未事先声明这两个过程。预期语义分析不会报告“未定义的过程”，验证内建过程 `read/write` 已在符号表中正确预注册。

2.2 错误测试（11 项）

testUndefinedIdentifier: 使用未声明的变量 `x` 进行赋值 `x := 1`。预期错误: "Undefined identifier: x"。验证标识符使用前必须声明。

testRedefinition: 在同一作用域内两次声明变量 `x`（均为 integer）。预期错误: "Redefinition of identifier: x"。验证同作用域内名字不可重复定义。

testAssignmentTypeMismatch: 声明 integer 变量，赋值为布尔字面量 `true`。预期错误: "Type mismatch in assignment"。验证赋值语句的左右类型必须兼容。

testProcedureCallArgCountMismatch: 过程 `p` 声明有 2 个参数，调用时仅传 1 个实参。预期错误: "Argument count mismatch in call to p"。验证过程调用实参数量须与形参一致。

testProcedureCallArgTypeMismatch: 过程 `p` 第一个参数为 `var x: integer`，调用时传入 `real` 类型的变量 `b`。预期错误: "Argument type mismatch for parameter 1 in call to p"。验证参数类型必须兼容。

testProcedureCallVarParamRequiresLValue: 过程 `p` 第一个参数为 `var` 引用参数，调用时传入字面量 `1`。预期错误: "var parameter requires assignable argument for parameter 1"。验证 `var` 参数要求实参为可赋值的左值。

testFunctionResultAssignmentInvalidOutsideFunction: 声明函数 `f`，在函数外部（全局作用域）执行 `f := 1`。预期错误: "Left-hand side of assignment is not assignable"。验证函数名仅在函数体内可作为赋值左值。

testArrayIndexOutOfBounds: 声明范围 1..3 的数组，使用常量下标 5 访问。预期错误: "Array index out of bounds"。验证编译期常量下标越界检测。

testArrayIndexConstExpressionOutOfBounds: 声明范围 1..3 的数组，使用常量表达式 1+3（求值得 4）作为下标。预期错误: "Array index out of bounds"。验证编译期常量表达式求值后的越界检测。

testMultiDimArrayBoundsCheck: 声明二维数组（外层 1..2，内层 10..12），使用有效外层下标 2 和越界内层下标 99 访问。预期错误: "Array index out of bounds"。验证多维数组中每个维度独立进行越界检查。

testProcedureCallCannotBeUsedAsValue: 声明过程 p，在表达式中将其调用作为值使用 a := p(a, b)。预期错误: "Procedure call cannot be used as a value"。验证无返回值的过程不可在值上下文中使用。

testProgramHeaderIdentifiersAreNotVariables: 程序头部声明 program example(input, output)，尝试赋值 input := 1。预期错误: "Undefined identifier: input"。验证程序头部参数名不被视为可使用的变量名。

3. 集成测试（合法用例）

所有语义集成测试均使用以下命令执行（--semantic 模式完成词法→语法→语义分析后停止）:

```
01 ./build/pascc -i <file.pas> --semantic
```

合法用例预期退出码 0，stdout 输出 Semantic analysis succeeded.。以下选取 6 个代表性用例。

3.1 多类型变量声明与赋值（sv01）

源码:

```
01 program sv01_basic_assign;
02 var
03     a, b: integer;
04     x: real;
05     flag: boolean;
06     ch: char;
07 begin
08     a := 1;
09     b := 2;
10     x := 3.14;
11     flag := true;
12     ch := 'c'
13 end.
```

语义分析输出：

```
01 Semantic analysis succeeded.
```

验证四种基本类型（integer、real、boolean、char）的变量声明和同类型赋值均通过类型检查。

3.2 过程调用与 var 参数（sv07）

源码：

```
01 program sv07_proc_call;
02 var a, b: integer;
03
04 procedure swap(var x, y: integer);
05 var tmp: integer;
06 begin
07     tmp := x;
08     x := y;
09     y := tmp;
10 end;
11
12 begin
13     a := 1;
14     b := 2;
15     swap(a, b);
16 end.
```

语义分析输出：

```
01 Semantic analysis succeeded.
```

验证 var 引用参数的声明和调用（传入变量 a, b 作为左值）正确通过参数检查。

3.3 阶乘递归函数（sv17）

源码：

```
01 program sv17_func_result;
02 var n, result: integer;
03
04 function factorial(x: integer): integer;
05 begin
06     if x <= 1 then
07         factorial := 1
08     else
09         factorial := x * factorial(x - 1)
10 end;
```

```
10 end;
11
12 begin
13     n := 5;
14     result := factorial(n)
15 end.
```

语义分析输出:

```
01 Semantic analysis succeeded.
```

验证递归函数声明、函数体内对函数名的赋值（返回值设置）、以及函数调用返回值赋给变量的全过程类型检查。

3.4 整数与实数混合运算 (sv19)

源码:

```
01 program sv19_mixed_arithmetic;
02 var
03     i: integer;
04     r, s: real;
05 begin
06     i := 3;
07     r := 2.5;
08     s := i + r;
09     s := i * r;
10     s := i / r;
11     s := r - i;
12     s := i / 2;
13     s := i + 1.0
14 end.
```

语义分析输出:

```
01 Semantic analysis succeeded.
```

验证整数与实数的混合算术运算 (+、-、*、/) 中，整数操作数可以隐式拓宽为实数，结果类型正确推导为 real。

3.5 非零起始数组 (sv21)

源码:

```
01 program sv21_arr_nonzero_start;
02 var
03     arr: array[3..9] of integer;
04     i, sum: integer;
```

```

05 begin
06     arr[3] := 1;
07     arr[9] := 7;
08     sum := 0;
09     for i := 3 to 9 do
10         sum := sum + arr[i]
11     end.

```

语义分析输出：

```

01 Semantic analysis succeeded.

```

验证声明范围 3..9 的数组，下标 3 和 9 均在声明范围内，for 循环变量 i 从 3 到 9 访问数组合法。

3.6 while 循环内 break (sv11)

源码：

```

01 program sv11_break_in_while;
02 var i: integer;
03 begin
04     i := 0;
05     while i < 100 do
06         begin
07             i := i + 1;
08             if i = 50 then
09                 break
10         end
11     end.

```

语义分析输出：

```

01 Semantic analysis succeeded.

```

验证 break 在 while 循环体内合法使用，语义分析器正确维护 loopDepth 计数器。

4. 集成测试（非法用例）

非法用例预期退出码非 0，stderr 输出具体错误信息。以下选取 11 个代表性用例，覆盖全部语义错误类别。

4.1 未声明变量 (si01)

源码：


```
01 program si01_undefined_var;  
02 begin  
03     x := 1  
04 end.
```

错误输出:

```
01 Error at 3:5 - Undefined identifier: x  
02 Error at 3:5 - Left-hand side of assignment is not assignable  
03 Error at 3:5 - Type mismatch in assignment: expected unknown, got integer
```

4.2 变量重复声明 (si02)

源码:

```
01 program si02_var_redefinition;  
02 var  
03     x: integer;  
04     x: boolean;  
05 begin  
06     x := 1  
07 end.
```

错误输出:

```
01 Error at 4:5 - Redefinition of identifier: x
```

4.3 赋值类型不匹配 (si06)

源码:

```
01 program si06_type_mismatch_assign;  
02 var a: integer;  
03 begin  
04     a := true  
05 end.
```

错误输出:

```
01 Error at 5:5 - Type mismatch in assignment: expected integer, got boolean
```

4.4 循环外 break (si11)

源码:

```
01 program si11_break_outside_loop;  
02 var x: integer;  
03 begin  
04     x := 1;
```

```
05     if x > 0 then
06         break
07 end.
```

错误输出:

```
01 Error at 7:9 - break can only be used inside while/for loop
```

4.5 数组下标越界 (si12)

源码:

```
01 program si12_array_index_bounds;
02 var arr: array[1..10] of integer;
03 begin
04     arr[0] := 1
05 end.
```

错误输出:

```
01 Error at 5:9 - Array index out of bounds: 0 not in [1, 10]
```

验证编译期常量下标 0 的越界检测, 给出实际值和声明范围的完整信息。

4.6 参数数量不匹配 (si15)

源码:

```
01 program si15_arg_count_mismatch;
02 var a: integer;
03 procedure add(x, y: integer);
04 begin a := x + y end;
05 begin
06     a := 0;
07     add(1)
08 end.
```

错误输出:

```
01 Error at 12:5 - Argument count mismatch in call to add: expected 2, got 1
```

4.7 var 参数传字面量 (si17)

源码:

```
01 program si17_var_param_literal;
02 procedure inc(var n: integer);
03 begin n := n + 1 end;
04 begin
```

```
05     inc(5)
06 end.
```

错误输出:

```
01 Error at 9:9 - var parameter requires assignable argument for parameter 1 in
call to inc
```

验证引用参数要求左值，传入字面量 5 被正确拒绝。

4.8 div 运算用于实数 (si18)

源码:

```
01 program si18_div_real;
02 var r: real;
03 begin
04     r := 3.0;
05     r := r div 2.0
06 end.
```

错误输出:

```
01 Error at 6:10 - div/mod operator requires integer operands
02 Error at 6:5 - Type mismatch in assignment: expected real, got unknown
```

4.9 过程调用用作表达式值 (si24)

源码:

```
01 program si24_proc_call_as_value;
02 var a, b: integer;
03 procedure getval;
04 begin write(1) end;
05 begin
06     a := getval
07 end.
```

错误输出:

```
01 Error at 11:5 - Type mismatch in assignment: expected integer, got procedure
```

验证过程（无返回值）不可在值上下文中使用。

4.10 read 读取常量 (si25)

源码:

```

01 program si25_read_non_lvalue;
02 const X = 1;
03 begin
04     read(X)
05 end.

```

错误输出:

```

01 Error at 5:10 - read expects assignable variable for parameter 1

```

验证内建过程 `read` 要求参数为可赋值的变量（左值），常量被正确拒绝。

4.11 实数缩窄赋值给整数（si27）

源码:

```

01 program si27_real_to_int_narrow;
02 var
03     i: integer;
04     r: real;
05 begin
06     r := 3.14;
07     i := r
08 end.

```

错误输出:

```

01 Error at 7:5 - Type mismatch in assignment: expected integer, got real

```

验证实数到整数的隐式缩窄转换不被允许（仅允许整数到实数的拓宽）。

5. 注解后 AST 示例

`--dump-annotated-ast` 标志在语义分析完成后输出注解后的 AST，展示每个节点的类型推导结果（`type`）、左值标记（`isLValue`）、符号表绑定（`[sym: {...}]`）以及数组边界信息（`[arrayBounds=...]`）。以下选取 5 个代表性用例。

5.1 基本声明与赋值（sv01）

```

01 pascc -i sv01_basic_assign.pas --dump-annotated-ast

```

```

01 Annotated AST:
02 Program (type: unknown)
03   Block (type: unknown)
04     ...
05     List (type: unknown)
06       VarDecl (type: unknown)

```

```

07     List (type: unknown)
08     Identifier (a, type: integer, isLValue: true)
09         [sym: {name=a, kind=variable, scope=0, type=integer}]
10     Identifier (b, type: integer, isLValue: true)
11         [sym: {name=b, kind=variable, scope=0, type=integer}]
12     Identifier (integer, type: integer, isLValue: false)
13 VarDecl (type: unknown)
14     List (type: unknown)
15     Identifier (x, type: real, isLValue: true)
16         [sym: {name=x, kind=variable, scope=0, type=real}]
17     Identifier (real, type: real, isLValue: false)
18 VarDecl (type: unknown)
19     List (type: unknown)
20     Identifier (flag, type: boolean, isLValue: true)
21         [sym: {name=flag, kind=variable, scope=0, type=boolean}]
22     Identifier (boolean, type: boolean, isLValue: false)
23 VarDecl (type: unknown)
24     List (type: unknown)
25     Identifier (ch, type: char, isLValue: true)
26         [sym: {name=ch, kind=variable, scope=0, type=char}]
27     Identifier (char, type: char, isLValue: false)
28 ...
29 CompoundStmt (type: unknown)
30     AssignStmt (type: unknown)
31         Identifier (a, type: integer, isLValue: true)
32             [sym: {name=a, kind=variable, scope=0, type=integer}]
33         Literal (type: integer)
34     AssignStmt (type: unknown)
35         Identifier (x, type: real, isLValue: true)
36             [sym: {name=x, kind=variable, scope=0, type=real}]
37         Literal (type: real)
38     AssignStmt (type: unknown)
39         Identifier (flag, type: boolean, isLValue: true)
40             [sym: {name=flag, kind=variable, scope=0, type=boolean}]
41         Literal (type: boolean)
42     AssignStmt (type: unknown)
43         Identifier (ch, type: char, isLValue: true)
44             [sym: {name=ch, kind=variable, scope=0, type=char}]
45         Literal (type: char)

```

每个 Identifier 节点在声明处和引用处均注解了推导出的类型（integer/real/boolean/char）和左值状态（变量为 true，类型名为 false）。Literal 节点直接标注字面量类型。符号表条目显示所有变量在全局作用域 scope=0。

5.2 过程调用与 var 参数 (sv07)

```
01 pascc -i sv07_proc_call.pas --dump-annotated-ast
```

```

01 Annotated AST:
02 Program (type: unknown)
03   Block (type: unknown)
04     ...
05     List (type: unknown)
06       ProcDecl (type: unknown)
07         List (type: unknown)
08           ParamDecl (type: unknown)
09             List (type: unknown)
10               Identifier (x, type: integer, isLValue: true)
11                 [sym: {name=x, kind=parameter, scope=1, type=integer,
isVarParam=true}]
12               Identifier (y, type: integer, isLValue: true)
13                 [sym: {name=y, kind=parameter, scope=1, type=integer,
isVarParam=true}]
14               Identifier (integer, type: integer, isLValue: false)
15             Block (type: unknown)
16               ...
17             CompoundStmt (type: unknown)
18               AssignStmt (type: unknown)
19                 Identifier (tmp, type: integer, isLValue: true)
20                   [sym: {name=tmp, kind=variable, scope=1, type=integer}]
21                 Identifier (x, type: integer, isLValue: true)
22                   [sym: {name=x, kind=parameter, scope=1, type=integer,
isVarParam=true}]
23               AssignStmt (type: unknown)
24                 Identifier (x, type: integer, isLValue: true)
25                   [sym: {name=x, kind=parameter, scope=1, type=integer,
isVarParam=true}]
26                 Identifier (y, type: integer, isLValue: true)
27                   [sym: {name=y, kind=parameter, scope=1, type=integer,
isVarParam=true}]
28               AssignStmt (type: unknown)
29                 Identifier (y, type: integer, isLValue: true)
30                   [sym: {name=y, kind=parameter, scope=1, type=integer,
isVarParam=true}]
31                 Identifier (tmp, type: integer, isLValue: true)
32                   [sym: {name=tmp, kind=variable, scope=1, type=integer}]
33             CompoundStmt (type: unknown)
34               ...
35             ProcCall (type: procedure)
36               [sym: {name=swap, kind=procedure, scope=0, type=procedure}]
37             Identifier (a, type: integer, isLValue: true)
38               [sym: {name=a, kind=variable, scope=0, type=integer}]
39             Identifier (b, type: integer, isLValue: true)
40               [sym: {name=b, kind=variable, scope=0, type=integer}]

```

局部变量 tmp 和形参 x/y 均在 scope=1 (swap 过程内部作用域), var 参数带有 isVarParam=true 标记。ProcCall 调用节点被注解为 type: procedure 并绑

定了 `swap` 的符号表条目，实参 `a/b` 各有类型注解。作用域层级（`scope 0` vs `1`）区分清晰。

5.3 数组访问与边界注解（sv10）

```
01 pascc -i sv10_array_access.pas --dump-annotated-ast
```

```
01 Annotated AST:
02 Program (type: unknown)
03   Block (type: unknown)
04     ...
05     List (type: unknown)
06       VarDecl (type: unknown)
07         List (type: unknown)
08           Identifier (arr, type: integer, isLValue: true)
09             [sym: {name=arr, kind=variable, scope=0, type=integer,
isArray=true}]
10             [arrayBounds=[1..10]]
11             ArrayType (type: integer)
12             Literal (type: unknown)
13             Literal (type: unknown)
14             Identifier (integer, type: unknown, isLValue: false)
15             ...
16       CompoundStmt (type: unknown)
17         AssignStmt (type: unknown)
18           ArrayAccess (type: integer)
19             [sym: {name=arr, ..., isArray=true}] [arrayBounds=[1..10]]
20             Identifier (arr, type: integer, isLValue: true)
21             [sym: {name=arr, ..., isArray=true}] [arrayBounds=[1..10]]
22             Literal (type: integer)
23             Literal (type: integer)
24             ...
25         AssignStmt (type: unknown)
26           Identifier (i, type: integer, isLValue: true) ...
27           BinaryExpr (type: integer)
28             ArrayAccess (type: integer) [arrayBounds=[1..10]]
29               Identifier (arr, type: integer, ...) [arrayBounds=[1..10]]
30               Literal (type: integer)
31             ArrayAccess (type: integer) [arrayBounds=[1..10]]
32               Identifier (arr, type: integer, ...) [arrayBounds=[1..10]]
33               Literal (type: integer)
```

数组变量 `arr` 带有 `isArray=true` 标记和 `[arrayBounds=[1..10]]` 边界注解。`ArrayAccess` 节点推导了元素类型 `type: integer` 并绑定了数组符号条目。二元加法表达式 `arr[1] + arr[5]` 的结果类型正确推导为 `integer`。

5.4 递归函数：if-else 与返回值赋值（sv17）

```
01 pascc -i sv17_func_result.pas --dump-annotated-ast
```

```
01 Annotated AST:
02 Program (type: unknown)
03   Block (type: unknown)
04     ...
05     List (type: unknown)
06       FuncDecl (type: unknown)
07         List (type: unknown)
08           ParamDecl (type: unknown)
09             List (type: unknown)
10               Identifier (x, type: integer, isLValue: true)
11                 [sym: {name=x, kind=parameter, scope=1, type=integer}]
12               Identifier (integer, type: integer, isLValue: false)
13             Block (type: unknown)
14               ...
15               CompoundStmt (type: unknown)
16                 IfStmt (type: unknown)
17                   BinaryExpr (type: boolean)
18                     Identifier (x, type: integer, isLValue: true)
19                       [sym: {name=x, kind=parameter, scope=1, type=integer}]
20                     Literal (type: integer)
21                   AssignStmt (type: unknown)
22                     Identifier (factorial, type: integer, isLValue: false)
23                       [sym: {name=factorial, kind=function, scope=0, type=integer}]
24                     Literal (type: integer)
25                   AssignStmt (type: unknown)
26                     Identifier (factorial, type: integer, isLValue: false)
27                       [sym: {name=factorial, kind=function, scope=0, type=integer}]
28                   BinaryExpr (type: integer)
29                     Identifier (x, type: integer, ...)
30                     ProcCall (type: integer)
31                       [sym: {name=factorial, kind=function, scope=0,
type=integer}]
32                   BinaryExpr (type: integer)
33                     Identifier (x, type: integer, ...)
34                     Literal (type: integer)
35                 CompoundStmt (type: unknown)
36                   ...
37                   AssignStmt (type: unknown)
38                     Identifier (result, type: integer, isLValue: true) ...
39                     ProcCall (type: integer)
40                       [sym: {name=factorial, kind=function, scope=0, type=integer}]
41                     Identifier (n, type: integer, isLValue: true) ...
```

关键观察:

- IfStmt 的条件表达式 $x \leq 1$ 被正确推导为 `type: boolean`。

- 函数名 `factorial` 在函数体内的赋值虽然 `isLValue: false` (符号表中是函数而非变量), 但语义分析器通过上下文栈 `functionContextStack_` 将其识别为合法的函数返回值赋值。
- 递归调用 `factorial(x - 1)` 被注解为 `ProcCall (type: integer)`, 因为函数返回 `integer` 类型。
- 外层 `result := factorial(n)` 中 `ProcCall` 同样推导为 `type: integer`。

5.5 错误用例：类型不匹配时的注解 (si06)

非法用例在语义分析报错的同时仍然输出注解后的 AST。

```
01 pascc -i si06_type_mismatch_assign.pas --dump-annotated-ast
```

```
01 Annotated AST:
02 Program (type: unknown)
03   Block (type: unknown)
04     ...
05     CompoundStmt (type: unknown)
06       AssignStmt (type: unknown)
07         Identifier (a, type: integer, isLValue: true)
08           [sym: {name=a, kind=variable, scope=0, type=integer}]
09         Literal (type: boolean)
10 Error at 5:5 - Type mismatch in assignment: expected integer, got boolean
```

即使存在类型错误, AST 仍被完整注解: 左侧 `a` 已绑定符号并推导为 `type: integer`, 右侧 `true` 被识别为 `type: boolean` 的字面量。错误信息在 AST 之后输出, 语义分析器设计为尽最大努力注解 AST 后再报告所有错误。

6. 自动测试结果

以下为 test/run_semantic_tests.sh 的实际运行输出，作为全部 55 个用例（26 合法 + 29 非法）全部通过的证明。

```
01 =====
02 [semantic] running valid cases...
03 =====
04 [semantic][valid] sv01_basic_assign
05 [semantic][valid] sv02_int_to_real_widening
06 [semantic][valid] sv03_if_statement
07 [semantic][valid] sv04_while_loop
08 [semantic][valid] sv05_for_to_loop
09 [semantic][valid] sv06_for_downto_loop
10 [semantic][valid] sv07_proc_call
11 [semantic][valid] sv08_func_call
12 [semantic][valid] sv09_var_param
13 [semantic][valid] sv10_array_access
14 [semantic][valid] sv11_break_in_while
15 [semantic][valid] sv12_break_in_for
16 [semantic][valid] sv13_unary_operators
17 [semantic][valid] sv14_relational_ops
18 [semantic][valid] sv15_div_mod
19 [semantic][valid] sv16_logical_ops
20 [semantic][valid] sv17_func_result
21 [semantic][valid] sv18_read_write
22 [semantic][valid] sv19_mixed_arithmetic
23 [semantic][valid] sv20_const_decl
24 [semantic][valid] sv21_arr_nonzero_start
25 [semantic][valid] sv22_proc_no_params
26 [semantic][valid] sv23_nested_if
27 [semantic][valid] sv24_complex_expr
28 [semantic][valid] sv25_multiple_vars
29 [semantic][valid] sv26_array_multidim
30
31 =====
32 [semantic] running invalid cases...
33 =====
34 [semantic][invalid] si01_undefined_var
35 [semantic][invalid] si02_var_redefinition
36 [semantic][invalid] si03_param_redefinition
37 [semantic][invalid] si04_proc_redefinition
38 [semantic][invalid] si05_func_redefinition
39 [semantic][invalid] si06_type_mismatch_assign
40 [semantic][invalid] si07_if_non_boolean
41 [semantic][invalid] si08_while_non_boolean
42 [semantic][invalid] si09_for_var_not_integer
43 [semantic][invalid] si10_for_bounds_not_int
```

```

44 [semantic][invalid] si11_break_outside_loop
45 [semantic][invalid] si12_array_index_bounds
46 [semantic][invalid] si13_array_index_not_int
47 [semantic][invalid] si14_subscript_non_array
48 [semantic][invalid] si15_arg_count_mismatch
49 [semantic][invalid] si16_arg_type_mismatch
50 [semantic][invalid] si17_var_param_literal
51 [semantic][invalid] si18_div_real
52 [semantic][invalid] si19_arith_on_boolean
53 [semantic][invalid] si20_unary_minus_bool
54 [semantic][invalid] si21_not_real
55 [semantic][invalid] si22_and_on_int
56 [semantic][invalid] si23_relational_mismatch
57 [semantic][invalid] si24_proc_call_as_value
58 [semantic][invalid] si25_read_non_lvalue
59 [semantic][invalid] si26_mod_real
60 [semantic][invalid] si27_real_to_int_narrow
61 [semantic][invalid] si28_const_redefinition
62 [semantic][invalid] si29_unary_minus_char
63
64 =====
65 [semantic] summary
66 =====
67 valid:    26/26
68 invalid:  29/29
69
70 All semantic tests passed.

```

7. 测试结果汇总

- **C++ 单元测试（正例）：**8 项，全部通过。
- **C++ 单元测试（负例）：**11 项，全部通过，各测试均对预期错误消息字符串进行了精确断言。
- **集成测试合法用例：**26 项，全部通过（`--semantic` 模式退出码 0）。
- **集成测试非法用例：**29 项，全部通过（`--semantic` 模式退出码非 0，`stderr` 包含语义错误信息）。
- **合计：**74 项测试全部通过，覆盖声明规则、类型检查、数组检查、过程/函数调用、控制流、内建过程等所有语义检查维度。